

Advanced Digital Design
130323
VHDL Source Simulation – Counter Simulation (Assignment 4)
Date: Tuesday, November 26, 2024

Submission:	By not more than 2 people together
Deadline: Final report	Tuesday, December 10, 2024 (17:15) – via moodle
According to JCT's policy, if you violate any submission or deadline criteria you disqualify your assignment	

A. Scope

This is another ModelSim exercise that walks you through VHDL source simulation, in order for you to get used to VHDL design, testbench, and to a few more ModelSim commands.

This exercise deals with a *free running 4-bit up counter, that uses an “active high” synchronous reset*. Counters are considered as a very important mechanism in Logic design. The predictive, step-by-step and predefined behavior of a counter makes it an ideal solution for logic control tasks. Moreover, the task of designing a counter in HDL languages is very simple and straightforward.

This assignment includes:

1. Step by step “hands on” exercise in sections B, C, D and E
2. Background and understanding questions in section F

B. Running ModelSim

1. Open a new directory in which you have read and write permissions. Copy the two VHDL files (cntr.vhd and tb_cntr.vhd), located in in the exercise directory into that new directory.
2. Open ModelSim by double clicking its icon located on the desktop.
3. Change directory to the new directory that you created in (1) above, by using the “File->”Change Directory...” menu in ModelSim.
4. Simulate the design by using the following ModelSim commands (Menu shortcuts are also possible):

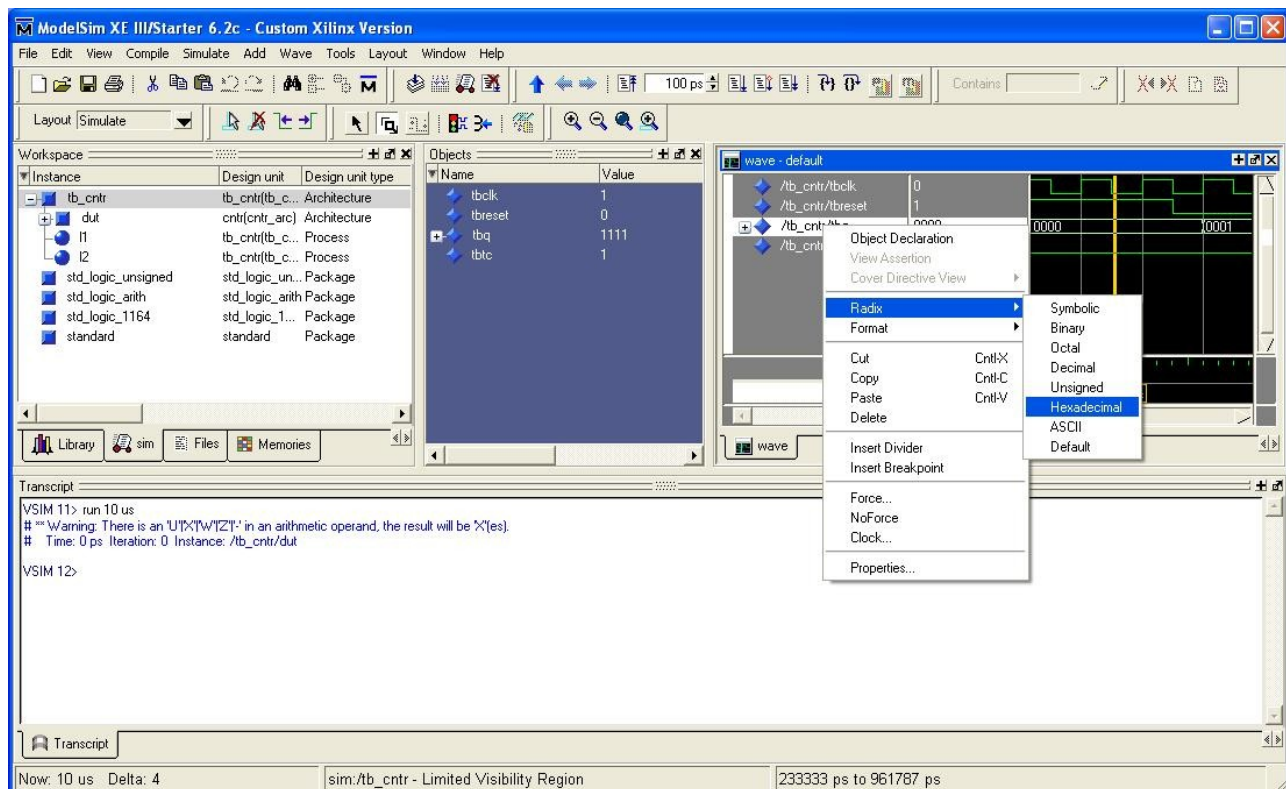
```
vlib work
vcom cntr.vhd
vcom tb_cntr.vhd
vsim -voptargs=+acc=lprn tb_cntr
```

5. Run the simulation for 4 us (microseconds). Use the simulator command:

```
run 4 us
```

Save this waveform for your final report (in your final report, explain what you see in the waveform and how does it prove that the counter design works correctly).

6. Change the radix (numeric base) that the counter's output in the testbench (tbQ) is presented in the waveform from Binary to hexadecimal. You need to do it by selecting/highlighting the signal and right-clicking the mouse. In the menu that will be open you need to select the “Radix” → “Hexadecimal” option. See figure 1 for more help:



Save the new waveform containing the new radix for your final report (in your final report, explain what is the advantage to see such a signal in a hexadecimal radix).

7. You can play with ModelSim by running and restarting the simulation a few times in order to get used for running such simulations in ModelSim.
8. We are going to learn a new way of maintaining waveforms in ModelSim more easily – by saving the waveform format. There are some differences in the exact way between different

versions of ModelSim, but the main approach is to select the “File → “Save Format” menu option from the waveform window (In some ModelSim versions there is only a “File” → “Save” option that leads to the same result). Save the format in your simulation directory, the file's default name is “wave.do”.

Please include this file (wave.do) in your report. Add some explanation in your report, in order to describe what you understand from this file.

9. Exit ModelSim by using the `quit -f` command.
10. Open ModelSim again, Change to your simulation directory and load the simulation again by running the related ModelSim command:
`vsim -voptargs=+acc=lprn tb_cntr`

11. Now run the following ModelSim command

```
do wave.do
```

ModelSim will load the waveform signals format according to what is specified in the wave.do file.

Run ModelSim for 10 us. What is the radix that ModelSim uses to present the tbQ signal?

12. Restart ModelSim (by using “`restart -f`” command) as we are going to learn another thing in running ModelSim... See section D for more details.

C. How much time does it really take ModelSim to simulate 1 second?

Simulation is not the real hardware! it is just exercising a behavioral/functional model....

In order to find out how much time does it take the simulator to run a certain command, one must gain an access to a real time clock. ModelSim does not have a real time clock but the system (PC/Window) has. So... in order to access the system's real time clock one needs to issue commands that access this clock.

ModelSim commands are based on the TCL language. One of the TCL commands is

```
clock seconds
```

- ➡ The “clock” command presents the system's clock in seconds. You can try out this command by running it a few times, waiting a few seconds between each time that you run the command.

So, what you need to do is to run the “clock seconds” command before and after you run the actual

simulation (of say, run 1000 ms). The best way to have the 3 commands run continuously is to write them in a script (“do file”) and run the script with the do command.

- The exercise directory contains a run.do file. Copy this file from the exercise directory to your simulation directory (open this file in an editor to make sure that the file contains the “clock” commands and run it by using the command

```
do run.do
```

Write the results (how much time does it take to run 1 second) in your report.

- We encourage some self understanding and thinking... What is the simulated frequency as specified in the testbench? How many clock cycles were simulated? Can you express the real run time by using a “real frequency”.

D. Experiment: Simulation with optimization

Note: This experiment can **only** be performed successfully if you use the “Full Featured” ModelSim simulator that is installed in the lab.

(I.e., don’t use the free version of “ModelSim” for this experiment).

1. Create a side directory, compile the “up counter” and the testbench with optimization:

```
vlib work
vcom cntr.vhd
vcom tb_cntr.vhd
vsim tb_cntr
```

Run the simulation with waveforms

2. Are there any signals that are not visible anymore? Which ones? (The answer to this question may depend on the version of the ModelSim software that you are using).

E. Simulation of the down counter:

1. Edit the counter's VHDL design and change it to a "down counter".

Notes:

- Down counter counts backwards (unlike an up counter, that counts forward)
 - In a "down counter", the counter ("re")sets to all ones and the TC is set when the counter is zero
2. Create a side directory, copy the "down counter" and the testbench (same testbench of the "up counter") to it.
 3. Compile the down counter and its testbench.
 4. Simulate the "down counter" for 5 us.

Include the waveform in your final report.

F. Background and Understanding:

The "Background and understanding" intention is to let you think and understand the counter's principles more thoroughly. The nature of the questions requires thinking, some questions were learned in previous courses (or maybe never learned before, so either the "common sense" or "Google" may be your best friends here).

This assignment is based on two VHDL files - `cntr.vhd` and `tb_cntr.vhd`, located in the exercise directory.

1. Go over these two files and explain each VHDL structure in these files (i.e., process, statement, signal assignments and more) according to what you have already learned in VHDL lectures 3 and 4, and according to previous assignments given in this course.
2. What is the clock frequency in which the testbench drives the design?
3. What is the range of counting given in `cntr.vhd`? Why?
4. Explain/define the following terms (feel free to search in Google):
 - "free running" counter
 - "stuck at" counter
 - "up" counter
 - "down" counter
5. What is the reset type (synchronous/asynchronous ?) and reset polarity (active high / active low ?) used in this counter design given in `cntr.vhd`? Explain!
6. Based on your answers to questions 1 – 5 write a "Spec" for the design given in `cntr.vhd` file (the spec should not be more than a few lines).
7. Write an ATP for the design given in `cntr.vhd` (the ATP should not be more than a few lines).

G. Your final report must include the followings:

1. Waveform of **B. 5**
2. Waveform of **B. 6**
3. The wave.do file, as required in **B.8**. Including some explanation describing what you understand from this file.
4. Answer to D.2
5. Waveform of the down counter simulation, as indicated in section E.
6. Results of measuring ModelSim's run time, as required in section D of this assignment:
 - How much time does it take to run 1 second?
 - What is the simulated frequency as specified in the testbench? How many clock cycles were simulated? Can you express the real run time by using a “real frequency”.
7. Answers to the background questions given in section F